Formally Verified Code Generation via Petri Nets for Rust's Ownership and Lifetimes

Kaiwen Zhang
Guanjun Liu*
zhangkw@tongji.edu.cn
liuguanjun@tongji.edu.cn
Tongji University
Shanghai, Shanghai, China

Abstract

Rust libraries increasingly expose stateful, type-directed APIs whose effective testing requires generating long, well-typed call sequences rather than mutating unstructured byte inputs. However, Rust's affine ownership, borrow exclusivity, and lifetime constraints impose a compilation barrier: naively generated sequences frequently fail to type-check due to move-after-use, illegal aliasing under &mut, or unsatisfied generic bounds, preventing any runtime feedback. This paper addresses signature-level test generation for Rust crates by modeling API invocations as a resource-flow problem over typed values and permissions.

We propose a Pushdown Colored Petri Net that is constructed from a crate's extracted function signatures. Places denote Rust type expressions and related compile-time obligations including generic placeholders, trait satisfaction, associated types, and field projections, while transitions capture signature-driven function calls and a small set of structural transformations. Token flow represents ownership transfer and borrow-induced capability changes, and a pushdown stack enforces properly nested borrow extents, binding produced references to their provenance and restoring permissions when borrows end. To support generic-heavy APIs, we define a partial-order-based satisfaction relation that checks whether a concrete instantiation meets trait bounds and associated-type constraints during transition enabling. Composite types are handled via formal projection and re-borrow transitions that preserve Rust's aliasing restrictions at the level of generated harness code. On top of the model, we formulate generation as a reachability problem with explicit coverage objectives over transitions and develop a bounded reachability analysis that searches for high-coverage firing traces under guard constraints.

Each trace is deterministically translated into a Rust harness, inserting drop or scope boundaries to realize stack-pop events, yielding programs that are compilable under the model's conservative borrowing discipline. We implement a prototype tool that integrates with the Rust toolchain for signature extraction and produces harnesses compatible with standard coverage-guided fuzzing workflows.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Rust, Colored Petri Nets, Formal Modeling, Fuzzing, Type Systems, API Sequence Generation

ACM Reference Format:
Kaiwen Zhang and Guanjun Liu. 2018. Formally Verified Code Generation via Petri Nets for Rust's Ownership and Lifetimes. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Rust has become a mainstream choice for building reliable systems software and performance-critical libraries. Its combination of low-level performance and a rich type system eliminates broad classes of memory-safety bugs, while still exposing fine-grained control over resources. As a result, an increasing amount of security and correctness-critical functionality is delivered as reusable Rust crates. For such libraries, automated test generation especially coverage-guided fuzzing, is an indispensable technique for discovering panics, logic bugs, and unexpected corner cases that escape conventional unit tests.

Despite its success, effective test-case generation for Rust libraries remains substantially harder than for many other languages. Modern fuzzers excel at mutating byte arrays and driving a single entry function, but library APIs are often stateful and type-directed: coverage typically depends on constructing a web of interacting objects and invoking a long sequence of API calls that respects resource protocols. In Rust, such protocols are enforced not only by runtime checks, but also by

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

compile-time rules on ownership, borrowing, and lifetimes. Consequently, a large fraction of naïvely generated call sequences never reach execution: they fail to compile due to move errors, aliasing violations, or unsatisfied generic constraints. This compilation barrier turns into a coverage barrier.

A motivating example. Consider a simplified crate API in Listing 1, which resembles many real-world libraries (parsers, databases, protocol stacks, etc.). It features (i) composite types (Result, Option, Vec), (ii) lifetime-carrying handles that borrow from a parent object, and (iii) a generic entry point constrained by trait bounds.

```rust
pub struct Store { /* ... */ }
pub struct Tx<'s> { store: &'s mut Store /* ... */ }
pub trait KeyLike {
    fn as_bytes(&self) -> &[u8];
}

impl Store {
    pub fn open(path: &str) -> Result<Self, ()> { /* ... */ }
    // Borrowing & lifetime: a transaction borrows the store mutably.
    pub fn begin<'s>(&'s mut self) -> Tx<'s> { /* ... */ }
}

impl<'s> Tx<'s> {
    pub fn put<K: KeyLike>(&mut self, k: K, v: Vec<u8>) { /* ... */ }
    // Immutable borrow from &self, returns a reference tied to 's.
    pub fn get<'a, K: KeyLike>(&'a self, k: K) -> Option<&'a [u8]> { /* ... */ }
    // Ownership consumption: committing consumes the Tx value.
    pub fn commit(self) -> Result<(), ()> { /* ... */ }
}
```

Listing 1: A small Rust API exhibiting ownership transfer, borrowing, lifetimes, and generics.

A test generator that ignores Rust's ownership discipline might attempt to interleave calls in ways that are invalid in Rust. For instance, after creating a transaction tx that holds &mut store, a generator may try to call store.begin() again, or mutate store directly, while tx is still live—both are rejected by the borrow checker. Similarly, commit(self) consumes tx, so any subsequent use of tx triggers a move error. Finally, generic parameters such as K: KeyLike require the generator to produce a concrete type that satisfies the bound, rather than an arbitrary byte blob. These issues are not merely syntactic. In Rust, well-typedness encodes a large portion of the API protocol. Therefore, achieving high coverage for Rust libraries hinges on the ability to generate compilable and type-correct call sequences that respect ownership, borrowing, lifetimes, and generic constraints.

A wide range of automated testing techniques exist, including coverage-guided fuzzing, property-based testing, grammar-based generation, symbolic execution, and model-based testing. However, when applied to Rust library APIs, they commonly face at least one of the following limitations:

1. Byte-oriented inputs without typed object construction: Many fuzzing setups operate on byte arrays and rely on a hand-written harness to translate bytes into structured objects. For stateful APIs, the harness must additionally manage long call sequences, intermediate values, and resource protocols. This manual effort is error-prone and often becomes the dominant bottleneck.
2. Limited support for Rust-specific resource constraints: Generators that treat values as freely copyable struggle with Rust's affine ownership: values can be moved, partially moved, borrowed immutably, or borrowed mutably, with strict aliasing rules. Moreover, Rust lifetimes are typically inferred such as non-lexical lifetimes, which means that the precise end of a borrow is not explicitly written in the source code, yet it is essential for deciding when subsequent mutations become legal.
3. Weak handling of composite and generic APIs: Real crates expose APIs over nested algebraic data types, slices, smart pointers, and rich trait-bound generics. Without a principled mechanism for instantiating generics and ensuring that candidate types satisfy trait bounds, generators either (i) avoid such functions, or (ii) produce ill-typed code that never compiles.
4. Poor scalability when internal state is over-modeled: Some analysis-driven approaches attempt to model the state of the internal program to drive generation. Although powerful, these approaches can be expensive and brittle for large crates and are often unnecessary when the goal is the generation of API-level tests rather than the verification of the full program

A parallel line of work explores Petri nets and related automata for generating event sequences and testing protocols. Classic Petri-net encodings, however, are typically untyped or only weakly typed: tokens represent abstract resources, but not Rust values with ownership and borrow capabilities. As a result, straightforward Petri-net-driven sequence generation tends to violate Rust's compile-time rules in three recurring ways: (i) it fails to enforce affinity such as a moved token must not be reused, (ii) it cannot express borrow exclusivity such as no mutable aliasing, no mutation under an outstanding immutable borrow, and (iii) it lacks a notion of scope and lifetime bounded availability, references must not outlive the borrowed owner, and borrow permissions must eventually return. Furthermore, generic constraints require a solver-like step selecting admissible instantiations, which traditional Petri-net models do not provide.

This paper targets a common and practically important setting: we are given a Rust codebase, and we aim to generate test harnesses that invoke its public functions and methods. Crucially, we assume that function signatures including input/output types, reference mutability, lifetimes, and generic bounds are available from the cargo doc. We do not attempt to precisely model the internal control flow or memory state of function bodies. Instead, we treat each function as an atomic transition whose type signature specifies how values and their capabilities can flow across call boundaries. This signature-level stance makes the approach lightweight and scalable, while still capturing the constraints that dominate compilation success for generated tests.

We introduce a new model for Rust test-case generation based on Pushdown Colored Petri Nets (PCPNs). At a high level: Places represent pools of available typed values together with capabilities describing how they may be used (owned, immutably borrowed, mutably borrowed). Transitions represent function calls extracted from the target crate's signatures. A transition consumes tokens matching the function's input types/capabilities and produces tokens matching its output types/capabilities. Colors encode Rust types (including composites) and capability metadata such as owned T, &T, &mut T. A pushdown stack models nested scopes and borrow extents. When a reference is created, the model pushes the borrowed provenance; when the reference becomes dead such as explicitly drop'd or out of scope, the model pops the corresponding frame and returns permissions to the original owner.

This pushdown structure is essential for Rust-style lifetime reasoning: borrow relationships can nest (re-borrowing), and long API sequences may create and discharge borrows repeatedly. A finite-state abstraction often either becomes overly conservative blocking legal programs or overly permissive. PCPNs provide a compact, compositional way to represent potentially unbounded, properly nested borrow patterns.

To support generic APIs, we equip the model with a type satisfaction relation written as a partial order $\preceq$. Intuitively, $A \preceq \mathcal{R}$ means that the concrete type A satisfies the generic requirement $\mathcal{R}$ such as a set of trait bounds, lifetime constraints, and where-clauses. During transition firing, generic parameters are instantiated by selecting concrete types that are minimal or otherwise prioritized under $\preceq$ while remaining consistent with the surrounding token colors. This design cleanly separates: (i) the Petri-net execution semantics for sequencing, and (ii) the constraint reasoning needed to produce admissible generic instantiations. In practice, the partial order enables both sound pruning and guided exploration.

From a PCPN execution trace, we synthesize a Rust harness that composes API calls and feeds the resulting program to a standard coverage-guided fuzzer. Listing 2 illustrates a typical generated skeleton for the API in Listing 1. The harness is compilable by construction: every use respects move semantics, borrow exclusivity, and lifetime boundaries.

```
fn harness(bytes: &[u8]) {
    // Construct base values (from bytes or small generators).
    let path = "tmp.db"; // tmp.db from Constant
    let mut store = match Store::open(path) { Ok(s) => s, Err(_) => return };
    // Begin a transaction: tx borrows store mutably.
    let mut tx = store.begin();
    // Choose a concrete key type K that satisfies K: KeyLike.
    // Here we pick Vec<u8> (assume the crate provides impl KeyLike for Vec<u8>).
    let k: Vec<u8> = bytes.to_vec();
    // Use tx while it is live; store cannot be used mutably here.
    tx.put(k.clone(), bytes.to_vec());
    let _v = tx.get(k);
    // Explicitly end the borrow extent and consume tx.
    let _ = tx.commit();
    // After commit, permissions return: store is usable again.
    let _tx2 = store.begin();
}
```

Listing 2: A compilable, signature-driven harness sketch generated from a PCPN trace.

Importantly, the generator does not need to model what Store::open, Tx::put, or Tx::get do internally. The signature alone determines how ownership and borrow capabilities move across call boundaries, which is exactly the information required to produce executable tests.

This paper makes the following contributions:

1. We formulate signature-driven Rust test generation as a typed resource-flow problem and propose PCPNs as a unifying model for generating compilable API call sequences under Rust ownership, borrowing, and lifetime constraints.
2. We design a capability-and-lifetime encoding for Rust values in PCPNs, including composite types and reborrowing patterns, and show how pushdown stack discipline captures nested borrow extents with returned permissions.
3. We introduce a partial-order-based mechanism for generic instantiation that checks whether concrete types satisfy trait-bound requirements, enabling exploration of generic-heavy APIs without manual stubs.
4. We evaluate the method on a diverse set of real-world crates and demonstrate that PCPN-driven generation substantially increases the fraction of compilable generated tests and improves coverage for stateful APIs that are challenging for byte-oriented harnesses.

Section 2 reviews Rust features that matter for test generation and distills the key constraints imposed by ownership, borrowing, lifetimes, and generics. Section **??** presents the PCPN model and its capability semantics. Section **??** describes how

we extract signatures, construct PCPNs, and synthesize compilable harnesses. Section 6 evaluates the approach on real-world crates and coverage-guided fuzzing workflows. Section **??** discusses related work in fuzzing, model-based testing, and Petri-net-driven sequence generation. Section **??** concludes and outlines future directions, including richer trait reasoning, environment modeling, and feedback-directed net refinement.

2 Background

2.1 Rust Ownership, Borrowing, and Lifetimes

Rust enforces memory safety primarily through a static discipline of ownership and borrowing. Every non-Copy value has exactly one owner at a time; transferring a value to another variable typically moves ownership. When the owner goes out of scope, the value is dropped and its resources are released.

Borrowing allows temporary access to a value without transferring ownership. Rust distinguishes: (i) shared borrows `&T`, which permit read-only access and can be aliased arbitrarily, and (ii) mutable borrows `&mut T`, which permit mutation but must be exclusive. The central rule is often summarized as aliasing XOR mutability: at any point, either many shared borrows exist, or exactly one mutable borrow exists.

A lifetime is a compile-time region that bounds how long a reference may be used. A reference cannot outlive the value it refers to, and borrow permissions return to the owner when the borrow ends. In modern Rust, borrow extents are often inferred via non-lexical lifetimes, meaning the precise end point of a borrow is not necessarily obvious from surface syntax, but is still crucial for deciding whether later mutations are legal.

For test-case generation, these rules impose constraints on call sequences: a generator must track whether a value is still owned, has been moved, is currently shared-borrowed, or is mutably borrowed, and it must ensure that references do not outlive their provenance.

2.2 Generics and Trait Bounds in Signatures

Rust APIs are frequently generic. A function may quantify over type parameters and constrain them with trait bounds possibly in `where` clauses. For example, a signature `fn f<T: Trait>(x: T) -> U` restricts admissible instantiations of `T` to types implementing `Trait`. From the perspective of automated test generation, this means that producing a call is a two-step task: (1) choose concrete instantiations for generic parameters, and (2) supply values whose types match the instantiated signature. If step (1) fails, the generated program will not type-check, regardless of runtime behavior.

3 Pushdown Colored Petri Nets for Rust API Test Generation

3.1 Problem Setting

Let $\mathcal{C}$ be a Rust crate. We can extract, from the compiler front-end, a finite set of *callable items* $\mathcal{F}$ (free functions and methods) together with their signatures, including (i) input/output types, (ii) reference mutability, (iii) explicit lifetime parameters and outlives constraints when present, and (iv) generic parameters with trait bounds and associated-type constraints.

Our goal is to generate compilable Rust harnesses that construct typed values and execute sequences of calls to items in $\mathcal{F}$. We deliberately avoid modeling internal states of function bodies; instead, each call is treated as an atomic transformation whose compile-time validity is determined by the signature-level ownership/borrowing/lifetime/generic constraints.

3.2 Petri Net Preliminaries

Definition 1 (Net and Petri Net). *A net is a 3-tuple $N = (P, T, F)$, where P is a finite set of places, T is a finite set of transitions, $F \subseteq (P \times T) \cup (T \times P)$ is a finite set of arcs, and $P \cap T = \emptyset$. A net can be viewed as a directed bipartite graph where places are represented by circles and transitions by rectangles. The pre-set and post-set of a node $x \in \{P \cup T\}$ are defined as ${}^\bullet x = \{y \in P \cup T \mid (y, x) \in F\}$ and $x^\bullet = \{y \in P \cup T \mid (x, y) \in F\}$, respectively. In this paper, it is assumed that the weight of each directed arc is 1.*

A marking of $N = (P, T, F)$ is a mapping $M : P \to \mathbb{N}$ where $M(p)$ is the number of tokens in place p. A net N with an initial marking M_0 is called a Petri Net and denoted as (N, M_0). Transition t is enabled at M if $\forall p \in {}^\bullet t : M(p) > 0$, denoted as $M[t\rangle$. A marking M is a deadlock marking if any transition is disabled at M. Firing an enabled transition t yields a new marking M': $M'(p) = M(p) - 1$ if $p \in {}^\bullet t \backslash t^\bullet$; $M'(p) = M(p) + 1$ if $p \in t^\bullet \backslash {}^\bullet t$; and otherwise $M'(p) = M(p)$. This is denoted as $M[t\rangle M'$. A marking M_k is reachable from M if $M_k = M$ or there exists a non-empty transition sequence $\sigma = t_1 t_2 \dots t_k$ such that $M[t_1\rangle M_1[t_2\rangle \dots M_{k-1}[t_k\rangle M_k$ (abbreviated as $M[\sigma\rangle M_k$). All markings reachable from M_0 in a Petri net is written as $\mathcal{M}$. This paper only considers bounded Petri nets, i.e., $\exists k \in \mathbb{N}, \forall M \in \mathcal{M} : M(p) \leq k$.

3.3 Pushdown Colored Petri Nets (PCPN)

Let GTy be the set of monomorphic Rust types extracted from $\mathcal{C}$. Let TVar be a set of generic type variables and LVar a set of lifetime variables. Let $\Sigma(\mathcal{C})$ denote the extracted signature/type environment.

We use the following type-expression language:

$$\tau \in \mathsf{Ty} ::= g \mid \alpha \mid \tau_0\langle\tau_1, \dots, \tau_n\rangle \mid (\tau_1, \dots, \tau_n) \mid [\tau] \mid \mathsf{Ref}^q_\ell(\tau) \mid \mathsf{Assoc}(\tau, \mathsf{Tr}, A) \mid \mathsf{Field}(\tau, f) \mid \mathsf{Impl}(\tau, \mathsf{Tr}),$$

where $g \in \mathsf{GTy}$, $\alpha \in \mathsf{TVar}$, $q \in \{\mathsf{shr}, \mathsf{mut}\}$, $\ell \in \mathsf{LVar}$, Tr ranges over traits, A over associated type names, and f over fields/projection paths.

Let $\ell_\bullet \in \mathsf{LVar}$ be a reserved lifetime variable that does not appear in extracted signatures. All internally created references are typed with $\ell_\bullet$ and carry the actual region label via $\lambda(\ell_\bullet)$.

Let $\mathsf{Cap} = \{\mathsf{own}, \mathsf{frz}, \mathsf{blk}, \mathsf{val}\}$. We define the place set:

$$P \;=\; \{\, p_{\kappa,\tau} \mid \kappa \in \mathsf{Cap},\ \tau \in \mathsf{Ty} \,\}.$$

Intuitively, $p_{\mathsf{own},\tau}$ stores owned values; $p_{\mathsf{frz},\tau}$ stores owned values frozen by shared borrows; $p_{\mathsf{blk},\tau}$ stores owned values blocked by a mutable borrow; $p_{\mathsf{val},\tau}$ stores non-owning values including references, witnesses, etc.

Let $\mathbb{V}$ be a countably infinite set of value identifiers (fresh Rust variable names), and $\mathbb{L}$ a countably infinite set of region labels. Let Θ be finite type substitutions $\theta : \mathsf{TVar} \rightharpoonup \mathsf{GTy}$, and Λ finite lifetime substitutions $\lambda : \mathsf{LVar} \rightharpoonup \mathbb{L}$.

A token color is

$$c \;=\; \langle v, \theta, \lambda \rangle \in \mathsf{Col} := \mathbb{V} \times \Theta \times \Lambda.$$

A colored marking is

$$M : P \to \mathbb{N}^{(\mathsf{Col})},$$

i.e., each place carries a finite multiset of colored tokens.

Pushdown stack. Let Γ be a stack alphabet:

$$\gamma ::= \mathsf{Freeze}(v) \mid \mathsf{Shr}(v_o, v_r, \ell) \mid \mathsf{Mut}(v_o, v_r, \ell),$$

where $\ell \in \mathbb{L}$ is a region label. A configuration is $\mathsf{Cfg} = \langle M, S \rangle$ where $S \in \Gamma^*$ (top at the left).

We model arc multiplicities by a weight function

$$W : (P \times T) \cup (T \times P) \to \mathbb{N}$$

with finite support, and let $F = \{(x, y) \mid W(x, y) > 0\}$. For a transition t, we write $W(p, t)$ and $W(t, p)$ for input/output multiplicities.

A *choice* for t at marking M is a multiset-valued function

$$\chi : P \to \mathbb{N}^{(\mathsf{Col})} \quad \text{s.t.} \quad \forall p \in P : \ \chi(p) \subseteq M(p) \wedge |\chi(p)| = W(p, t).$$

Let ι range over instantiations, containing at least:

$$\iota = (\theta^*, \lambda^*, \nu, \mu, \dots)$$

where $\theta^* \in \Theta$ and $\lambda^* \in \Lambda$ are the unified substitutions, and ν and μ provide any fresh value ids / fresh region labels needed by Out. Other resolved information such as associated-type resolution may also be stored.

Each transition t carries:

$$\mathsf{Guard}(t)(\langle M, S \rangle, \chi, \iota),$$
$$\mathsf{Out}(t)(\langle M, S \rangle, \chi, \iota) : P \to \mathbb{N}^{(\mathsf{Col})},$$
$$\mathsf{Act}(t) \in \{\epsilon, \mathsf{push}(\bar{\gamma}), \mathsf{pop}(\bar{\gamma})\}.$$

3.4 Generics and Constraint Solving

To keep the net bounded while still allowing generic instantiation, we fix a finite set $\mathsf{GTy}^\dagger \subseteq \mathsf{GTy}$ of ground Rust types considered by the generator. Let $\widehat{\mathsf{Ty}}$ be the finite closure of $\mathsf{GTy}^\dagger$ under the type constructors that we model, such as $\mathsf{Ref}^q_\ell(\cdot)$, tuples, arrays, and any finite set of crate-local projections forms, so that all runtime values produced by the net have types in $\widehat{\mathsf{Ty}}$.

For substitutions $\theta : \mathsf{TVar} \rightharpoonup \mathsf{GTy}^\dagger$ and $\lambda : \mathsf{LVar} \rightharpoonup \mathbb{L}$, let $(\theta, \lambda)(\tau)$ denote capture-avoiding simultaneous substitution on type expressions.

An obligation is one of:

$$o ::= \tau : \mathsf{Tr} \mid \mathsf{Assoc}(\tau, \mathsf{Tr}, A) = \tau' \mid \ell_1 : \ell_2,$$

and a constraint set ρ is a finite set of obligations.

We assume an entailment judgement $\Sigma(\mathcal{C}) \vdash o$ meaning obligation o is derivable from the crate. We write

$$\Sigma(\mathcal{C}) \vdash (\theta, \lambda) \vDash \rho \quad \text{iff} \quad \forall o \in \rho : \ \Sigma(\mathcal{C}) \vdash (\theta, \lambda)(o).$$

This captures supertraits, where-clauses, and associated-type equalities in a uniform way without explicit monomorphization.

Let $\mathsf{Unify}(\tau, \hat{\tau})$ attempt to unify a type scheme $\tau \in \mathsf{Ty}$ with a ground type $\hat{\tau} \in \widehat{\mathsf{Ty}}$ and return a substitution pair (θ, λ), or $\perp$ if unification fails. Unification is structural on the type-expression syntax including Ref and projections, and is merged across multiple arguments in the standard way to obtain a single (θ^*, λ^*) when possible.

A transition corresponding to a generic API is enabled only if there exists a grounding (θ^*, λ^*) such that: (i) all chosen argument tokens' types unify with the signature parameter types under (θ^*, λ^*); (ii) $\Sigma(\mathcal{C}) \vdash (\theta^*, \lambda^*) \vDash \rho$. This delays generic instantiation to firing time, avoiding up-front state explosion.

3.5 Constructing a Rust PCPN from $\mathcal{C}$

We construct a crate-specific PCPN

$$\mathcal{N}_{\mathcal{C}} = (P, T, W, \mathsf{Col}, \Gamma, M_0, \mathsf{Guard}, \mathsf{Out}, \mathsf{Act}),$$

where P is indexed by admissible ground types $\widehat{\mathsf{Ty}}$, and M_0 seeds a finite set of constructors producing tokens of types in $\widehat{\mathsf{Ty}}$.

We write $\mathsf{IsCopy}(\hat{\tau})$ iff $\Sigma(\mathcal{C}) \vDash \hat{\tau} : \mathsf{Copy}$.

For each callable item $f \in \mathcal{F}$ with signature

$$\mathsf{sig}(f) = \langle \bar{\alpha}; \bar{\ell};\ \tau_1, \dots, \tau_n \to \tau_o;\ \rho \rangle,$$

we add a transition t_f. The net structure of t_f only specifies argument modes, while the concrete ground types of consumed/produced places are determined by a satisfiable instantiation (θ^*, λ^*) in the guard.

Let $\hat{\tau}_i$ denote the ground type of the chosen argument token for parameter i. The mode constraints are:

- If parameter i is by-value τ_i, then the chosen token must come from $p_{\mathsf{own},\hat{\tau}_i}$. If $\mathsf{IsCopy}(\hat{\tau}_i)$, we model by-value passing as a self-loop (token reproduced).
- If parameter i is shared reference $\mathsf{Ref}^{\mathsf{shr}}_\ell(\tau)$, then the chosen token must come from $p_{\mathsf{val},\hat{\tau}_i}$ where $\hat{\tau}_i$ is a ground shared-ref type; and we add a self-loop since $\&T$ is Copy.
- If parameter i is mutable reference $\mathsf{Ref}^{\mathsf{mut}}_\ell(\tau)$, then the chosen token must come from $p_{\mathsf{val},\hat{\tau}_i}$ where $\hat{\tau}_i$ is a ground mutable-ref type. *Rust-like default: implicit reborrow.* We treat this parameter as a self-loop (the reference remains

usable after the call) *unless* the reference must be moved because its lifetime escapes through outputs.

Let $\mathsf{FV}_\ell(\tau)$ return the set of lifetime variables appearing in τ. For a mutable-ref parameter $\mathsf{Ref}^{\mathsf{mut}}_\ell(\tau)$, define

$$\mathsf{Esc}(\ell, \tau_o) \;\equiv\; \ell \in \mathsf{FV}_\ell(\tau_o).$$

If $\mathsf{Esc}(\ell, \tau_o)$ holds, we do not self-loop that &mut argument; otherwise, we self-loop it to model implicit re-borrow in ordinary calls.

$\mathsf{Guard}(t_f)$: (i) picks ground argument types $\hat{\tau}_1, \ldots, \hat{\tau}_n \in \widehat{\mathsf{Ty}}$ by selecting tokens from the corresponding typed places under the above mode constraints, (ii) computes a unified instantiation (θ^*, λ^*) such that $\forall i : \mathsf{Unify}(\tau_i, \hat{\tau}_i) \neq \bot$ and all unifiers are compatible, and (iii) checks $\Sigma(\mathcal{C}) \vdash (\theta^*, \lambda^*) \vDash \rho$.

$\mathsf{Out}(t_f)$ allocates fresh ids for returned values and reproduces tokens on self-looped arguments. The grounded return type is $\hat{\tau}_o = (\theta^*, \lambda^*)(\tau_o)$, required to be in $\widehat{\mathsf{Ty}}$. By default $\mathsf{Act}(t_f) = \epsilon$.

3.6 Ownership and Copy as Token Flow

Let $\pi_V(\langle v, \theta, \lambda\rangle) = v$, $\pi_\Theta(\cdot) = \theta$, and $\pi_\Lambda(\cdot) = \lambda$. For a singleton multiset $\{c\}$, write $\mathsf{one}(\{c\}) = c$.

(1) Move for non-Copy owned values. For each type τ, add a transition Move_τ with:

$$W(p_{\mathsf{own},\tau}, \mathsf{Move}_\tau) = 1,$$

and no outputs. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{Move}_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv \neg\mathsf{IsCopy}(\tau),\\
\mathsf{Out}(\mathsf{Move}_\tau)(\langle M, S\rangle, \chi, \iota) &= \emptyset,\\
\mathsf{Act}(\mathsf{Move}_\tau) &= \epsilon.
\end{aligned}$$

(2) By-value use for Copy owned values (self-loop). For each τ, add $\mathsf{CopyUse}_\tau$:

$$W(p_{\mathsf{own},\tau}, \mathsf{CopyUse}_\tau) = 1, \qquad W(\mathsf{CopyUse}_\tau, p_{\mathsf{own},\tau}) = 1.$$

Let $c = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{CopyUse}_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv \mathsf{IsCopy}(\tau),\\
\mathsf{Out}(\mathsf{CopyUse}_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{c\} \,],\\
\mathsf{Act}(\mathsf{CopyUse}_\tau) &= \epsilon.
\end{aligned}$$

Fix a per-type copy budget $B_{\mathsf{copy}}(\tau) \in \mathbb{N}$. Add $\mathsf{DupCopy}_\tau$:

$$W(p_{\mathsf{own},\tau}, \mathsf{DupCopy}_\tau) = 1, \qquad W(\mathsf{DupCopy}_\tau, p_{\mathsf{own},\tau}) = 2.$$

Let $c = \langle v, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$ and let $v(\mathsf{new}) = v'$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{DupCopy}_\tau)(\langle M, S\rangle, \chi, \iota) \equiv\; & \mathsf{IsCopy}(\tau)\\
& \wedge\; |M(p_{\mathsf{own},\tau})| < B_{\mathsf{copy}}(\tau)\\
& \wedge\; v' \notin \mathsf{Ids}(M, S),\\
\mathsf{Out}(\mathsf{DupCopy}_\tau)(\langle M, S\rangle, \chi, \iota) = [\, & p_{\mathsf{own},\tau} \mapsto \{\langle v, \theta, \lambda\rangle,\ \langle v', \theta, \lambda\rangle\} \,],\\
\mathsf{Act}(\mathsf{DupCopy}_\tau) = \epsilon.
\end{aligned}$$

(3) Drop. For each place $p_{\kappa,\tau}$, add $\mathsf{Drop}_{\kappa,\tau}$:

$$W(p_{\kappa,\tau}, \mathsf{Drop}_{\kappa,\tau}) = 1,$$

and no outputs. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{Drop}_{\kappa,\tau})(\langle M, S\rangle, \chi, \iota) &\equiv \top,\\
\mathsf{Out}(\mathsf{Drop}_{\kappa,\tau})(\langle M, S\rangle, \chi, \iota) &= \emptyset,\\
\mathsf{Act}(\mathsf{Drop}_{\kappa,\tau}) &= \epsilon.
\end{aligned}$$

(4) First shared borrow from an owned value. Add $\mathsf{BorrowShrOwn}_\tau$ as T_τ: $W(p_{\mathsf{own},\tau}, T_\tau)$=$W(T_\tau, p_{\mathsf{frz},\tau})$=$W(T_\tau, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)})$=1.

Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$ and let $v(\mathsf{ref}) = v_r$, $\mu(\mathsf{lab}) = \ell$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv v_r \notin \mathsf{Ids}(M, S) \;\wedge\; \ell \notin \mathsf{Labs}(M, S),\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{frz},\tau} \mapsto \{c_o\},\\
&\qquad p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)} \mapsto \{\langle v_r, \theta, \lambda[\ell_\bullet \mapsto \ell]\rangle\} \,],\\
\mathsf{Act}(T_\tau) &= \mathsf{push}(\mathsf{Shr}(v_o, v_r, \ell) \cdot \mathsf{Freeze}(v_o)).
\end{aligned}$$

(5) Additional shared borrow while already frozen. Add $\mathsf{BorrowShrFrz}_\tau$ as T_τ: $W(p_{\mathsf{frz},\tau}, T_\tau)$=$W(T_\tau, p_{\mathsf{frz},\tau})$=$W(T_\tau, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)})$=1. Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{frz},\tau}))$ and let $v(\mathsf{ref}) = v_r$, $\mu(\mathsf{lab}) = \ell$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv v_r \notin \mathsf{Ids}(M, S) \;\wedge\; \ell \notin \mathsf{Labs}(M, S),\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{frz},\tau} \mapsto \{c_o\},\\
&\qquad p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)} \mapsto \{\langle v_r, \theta, \lambda[\ell_\bullet \mapsto \ell]\rangle\} \,],\\
\mathsf{Act}(T_\tau) &= \mathsf{push}(\mathsf{Shr}(v_o, v_r, \ell)).
\end{aligned}$$

(6) Mutable borrow from an owned value. Add $\mathsf{BorrowMut}_\tau$ as T_τ: $W(p_{\mathsf{own},\tau}, T_\tau)$=$W(T_\tau, p_{\mathsf{blk},\tau})$=$W(T_\tau, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)})$=1. Let $c_o = \langle v_o, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$ and let $v(\mathsf{ref}) = v_r$, $\mu(\mathsf{lab}) = \ell$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv v_r \notin \mathsf{Ids}(M, S) \;\wedge\; \ell \notin \mathsf{Labs}(M, S),\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{blk},\tau} \mapsto \{c_o\},\\
&\qquad p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)} \mapsto \{\langle v_r, \theta, \lambda[\ell_\bullet \mapsto \ell]\rangle\} \,],\\
\mathsf{Act}(T_\tau) &= \mathsf{push}(\mathsf{Mut}(v_o, v_r, \ell)).
\end{aligned}$$

3.7 Lifetimes via Stack Binding

(7) Ending a mutable borrow (restore owner). Add EndMut_τ as T_τ: $W(p_{\mathsf{blk},\tau}, T_\tau)$=$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)}, T_\tau)$=$W(T_\tau, p_{\mathsf{own},\tau})$=1.

Let $c_o = \langle v_o, \theta_o, \lambda_o\rangle = \mathsf{one}(\chi(p_{\mathsf{blk},\tau}))$ and $c_r = \langle v_r, \theta_r, \lambda_r\rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv S = \mathsf{Mut}(v_o, v_r, \ell) \cdot S'\\
&\qquad \wedge\; \theta_o = \theta_r \;\wedge\; \lambda_o = \lambda_r,\\
\mathsf{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{\langle v_o, \theta_o, \lambda_o\rangle\} \,],\\
\mathsf{Act}(T_\tau) &= \mathsf{pop}(\mathsf{Mut}(v_o, v_r, \ell)).
\end{aligned}$$

(8) Ending a shared borrow (non-last). Add $\mathsf{EndShrKeep}_\tau$ as T_τ: $W(p_{\mathsf{frz},\tau}, T_\tau)$=$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{shr}}_{\ell_\bullet}(\tau)}, T_\tau)$=$W(T_\tau, p_{\mathsf{frz},\tau})$=1.

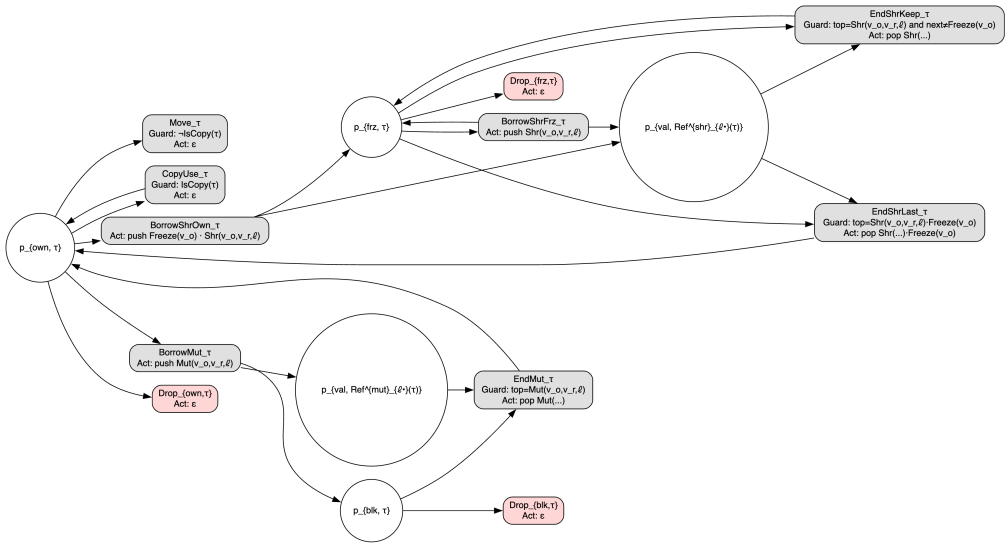

Figure 1: PCPN mapping: ownership + lifetimes (parametric in τ)

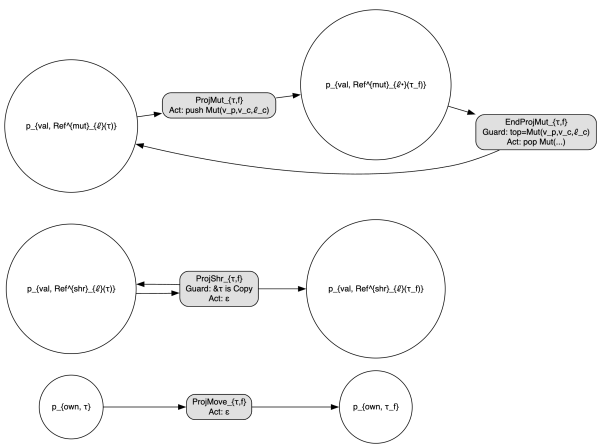

Figure 2: PCPN mapping: field projection & re-borrow (struct τ has field $f : \tau_f$)

Let $c_o = \langle v_o, \theta, \lambda \rangle = \mathrm{one}(\chi(p_{\mathrm{frz},\tau}))$ and $c_r = \langle v_r, \theta', \lambda' \rangle = \mathrm{one}(\chi(p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell_\bullet}(\tau)}))$. Define:

$$\begin{aligned}
\mathrm{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv S = \mathrm{Shr}(v_o, v_r, \ell) \cdot S' \\
&\quad \wedge\ \theta = \theta' \ \wedge\ \lambda = \lambda' \\
&\quad \wedge \big(S' = \epsilon \ \vee\ \mathrm{top}(S') \neq \mathrm{Freeze}(v_o)\big), \\
\mathrm{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathrm{frz},\tau} \mapsto \{\langle v_o, \theta, \lambda\rangle\} \,], \\
\mathrm{Act}(T_\tau) &= \mathrm{pop}(\mathrm{Shr}(v_o, v_r, \ell)).
\end{aligned}$$

(9) Ending a shared borrow (last, restore owner). Add $\mathrm{EndShrLast}_\tau$ as T_τ: $W(p_{\mathrm{frz},\tau}, T_\tau) = W(p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell_\bullet}(\tau)}, T_\tau) = W(T_\tau, p_{\mathrm{own},\tau}) = 1$.

Let $c_o = \langle v_o, \theta, \lambda \rangle = \mathrm{one}(\chi(p_{\mathrm{frz},\tau}))$ and $c_r = \langle v_r, \theta', \lambda' \rangle = \mathrm{one}(\chi(p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell_\bullet}(\tau)}))$. Define:

$$\begin{aligned}
\mathrm{Guard}(T_\tau)(\langle M, S\rangle, \chi, \iota) &\equiv S = \mathrm{Shr}(v_o, v_r, \ell) \cdot \mathrm{Freeze}(v_o) \cdot S' \\
&\quad \wedge\ \theta = \theta' \ \wedge\ \lambda = \lambda', \\
\mathrm{Out}(T_\tau)(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathrm{own},\tau} \mapsto \{\langle v_o, \theta, \lambda\rangle\} \,], \\
\mathrm{Act}(T_\tau) &= \mathrm{pop}(\mathrm{Shr}(v_o, v_r, \ell) \cdot \mathrm{Freeze}(v_o)).
\end{aligned}$$

3.8 Composite Types and Field Re-Borrowing

(10) Field projection by move. Let τ be a struct type with field $f : \tau_f$. Add $\mathrm{ProjMove}_{\tau,f}$ as $T_{\tau,f}$:

$$W(p_{\mathrm{own},\tau}, T_{\tau,f}) = W(T_{\tau,f}, p_{\mathrm{own},\tau_f}) = 1.$$

Let $c = \langle v, \theta, \lambda \rangle = \mathrm{one}(\chi(p_{\mathrm{own},\tau}))$ and let $v(\mathrm{fld}) = v_f$ be fresh. Define:

$$\begin{aligned}
\mathrm{Guard}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &\equiv v_f \notin \mathrm{Ids}(M, S), \\
\mathrm{Out}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathrm{own},\tau_f} \mapsto \{\langle v_f, \theta, \lambda\rangle\} \,], \\
\mathrm{Act}(T_{\tau,f}) &= \epsilon.
\end{aligned}$$

(11) Field reborrow from a shared reference (Copy, keep parent). Add $\mathrm{ProjShr}_{\tau,f}$ as $T_{\tau,f}$:

$$W(p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell}(\tau)}, T_{\tau,f}) = W(T_{\tau,f}, p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell}(\tau)}) = W(T_{\tau,f}, p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell}(\tau_f)}) = 1.$$

Let $c_p = \langle v_p, \theta, \lambda \rangle = \mathrm{one}(\chi(p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell}(\tau)}))$ and let $v(\mathrm{child}) = v_c$ be fresh. Define:

$$\begin{aligned}
\mathrm{Guard}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &\equiv v_c \notin \mathrm{Ids}(M, S), \\
\mathrm{Out}(T_{\tau,f})(\langle M, S\rangle, \chi, \iota) &= [\, p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell}(\tau)} \mapsto \{c_p\}, \\
&\qquad p_{\mathrm{val},\mathrm{Ref}^{\mathrm{shr}}_{\ell}(\tau_f)} \mapsto \{\langle v_c, \theta, \lambda\rangle\} \,], \\
\mathrm{Act}(T_{\tau,f}) &= \epsilon.
\end{aligned}$$

(12) Field reborrow from a mutable reference (reborrowing, block parent ref). Add $\mathsf{ProjMut}_{\tau,f}$ as $T_{\tau,f}$:

$$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell}(\tau)}, T_{\tau,f}) = W(T_{\tau,f}, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau_f)}) = 1.$$

Let $c_p = \langle v_p, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell}(\tau)}))$ and let $v(\mathsf{child}) = v_c$, $\mu(\mathsf{lab}) = \ell_c$ be fresh. Define:

$$\begin{aligned}
\mathsf{Guard}(T_{\tau,f})(\langle M,S\rangle,\chi,\iota) &\equiv v_c \notin \mathsf{Ids}(M,S) \ \wedge\ \ell_c \notin \mathsf{Labs}(M,S),\\
\mathsf{Out}(T_{\tau,f})(\langle M,S\rangle,\chi,\iota) &= [\, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau_f)} \mapsto \{\langle v_c,\theta,\lambda[\ell_\bullet \mapsto \ell_c]\rangle\}\,],\\
\mathsf{Act}(T_{\tau,f}) &= \mathsf{push}(\mathsf{Mut}(v_p, v_c, \ell_c)).
\end{aligned}$$

(13) Ending a mutable reborrow (restore parent ref). Add $\mathsf{EndProjMut}_{\tau,f}$ as $T_{\tau,f}$:

$$W(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau_f)}, T_{\tau,f}) = W(T_{\tau,f}, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell}(\tau)}) = 1.$$

Let $c_c = \langle v_c, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell_\bullet}(\tau_f)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(T_{\tau,f})(\langle M,S\rangle,\chi,\iota) &\equiv S = \mathsf{Mut}(v_p, v_c, \ell_c)\cdot S',\\
\mathsf{Out}(T_{\tau,f})(\langle M,S\rangle,\chi,\iota) &= [\, p_{\mathsf{val},\mathsf{Ref}^{\mathsf{mut}}_{\ell}(\tau)} \mapsto \{\langle v_p,\theta,\lambda\rangle\}\,],\\
\mathsf{Act}(T_{\tau,f}) &= \mathsf{pop}(\mathsf{Mut}(v_p, v_c, \ell_c)).
\end{aligned}$$

3.9 Traits and Associated Types

(14) Trait witness production. If $\Sigma(\mathcal{C}) \vDash \tau : \mathsf{Tr}$, add $\mathsf{Impl}_{\tau,\mathsf{Tr}}$:

$$W(p_{\mathsf{own},\tau}, \mathsf{Impl}_{\tau,\mathsf{Tr}}) = W(\mathsf{Impl}_{\tau,\mathsf{Tr}}, p_{\mathsf{own},\tau}) = 1$$

$W(\mathsf{Impl}_{\tau,\mathsf{Tr}}, p_{\mathsf{val},\mathsf{Impl}(\tau,\mathsf{Tr})}) = 1.$
Let $c = \langle v, \theta, \lambda\rangle = \mathsf{one}(\chi(p_{\mathsf{own},\tau}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(\mathsf{Impl}_{\tau,\mathsf{Tr}})(\langle M,S\rangle,\chi,\iota) &\equiv \Sigma(\mathcal{C}) \vDash \tau : \mathsf{Tr},\\
\mathsf{Out}(\mathsf{Impl}_{\tau,\mathsf{Tr}})(\langle M,S\rangle,\chi,\iota) &= [\, p_{\mathsf{own},\tau} \mapsto \{c\},\\
&\qquad p_{\mathsf{val},\mathsf{Impl}(\tau,\mathsf{Tr})} \mapsto \{\langle v,\theta,\lambda\rangle\}\,],\\
\mathsf{Act}(\mathsf{Impl}_{\tau,\mathsf{Tr}}) &= \epsilon.
\end{aligned}$$

(15) Associated type resolution as retyping. If $\Sigma(\mathcal{C}) \vDash \mathsf{Assoc}(\tau, \mathsf{Tr}, A) = \tau'$, add $\mathsf{AssocCast}_{\tau,\mathsf{Tr},A}$ as $T_{\tau,\mathsf{Tr},A}$:

$$W(p_{\mathsf{val},\mathsf{Assoc}(\tau,\mathsf{Tr},A)}, T_{\tau,\mathsf{Tr},A}) = W(T_{\tau,\mathsf{Tr},A}, p_{\mathsf{val},\tau'}) = 1.$$

Let $c = \mathsf{one}(\chi(p_{\mathsf{val},\mathsf{Assoc}(\tau,\mathsf{Tr},A)}))$. Define:

$$\begin{aligned}
\mathsf{Guard}(T_{\tau,\mathsf{Tr},A})(\langle M,S\rangle,\chi,\iota) &\equiv \Sigma(\mathcal{C}) \vDash \mathsf{Assoc}(\tau,\mathsf{Tr},A) = \tau',\\
\mathsf{Out}(T_{\tau,\mathsf{Tr},A})(\langle M,S\rangle,\chi,\iota) &= [\, p_{\mathsf{val},\tau'} \mapsto \{c\}\,],\\
\mathsf{Act}(T_{\tau,\mathsf{Tr},A}) &= \epsilon.
\end{aligned}$$

3.10 Soundness Intuition

By construction, any transition sequence in $\mathcal{N}_{\mathcal{C}}$ corresponds to a harness in which: (i) moved owned values are not reused, they are consumed from $p_{\mathsf{own},\tau}$, (ii) mutable aliasing is prevented. owners are moved to $p_{\mathsf{blk},\tau}$ under Mut frames, (iii) mutation under shared aliasing is prevented. owners reside in $p_{\mathsf{frz},\tau}$ under Freeze/Shr frames. and (iv) lifetimes are properly nested and ended through stack-pop transitions. Generic instantiation is guarded by the satisfiability relation $\Vdash$ and associated-type resolution. Therefore, generated harnesses are expected to be accepted by the Rust compiler under this conservative, structured borrowing discipline.

4 Bounded Reachability Analysis and Trusted Snippet Synthesis

This section presents a bounded analysis framework for the PCPN $\mathcal{N}_{\mathcal{C}}$. Places and token colors encode Rust resources together with typing and lifetime information. The pushdown stack encodes structured scopes induced by borrows and projections. The analysis constructs a finite canonical reachability graph under explicit budgets and canonicalization. Standard graph algorithms then decide reachability, deadness, liveness, and deadlocks inside the bounded domain. A witness firing sequence extracted from the graph is finally translated to a self-contained Rust snippet.

4.1 Configurations, enabledness, and firing

A configuration is a pair $\mathsf{Cfg} = \langle M, S\rangle$ where M is a marking over places and S is a stack word. The marking represents currently available symbolic resources. The stack represents open scopes that must be discharged to obtain a self-contained snippet.

Definition 2 (Enabledness and firing). *Let* $\mathsf{Cfg} = \langle M, S\rangle$. *A transition t is enabled at* Cfg *if there exist χ and ι such that* $\mathsf{Guard}(t)(\mathsf{Cfg}, \chi, \iota)$ *holds and the pop-pattern matches the stack top. A firing rule is $\varphi = \langle t, \chi, \iota\rangle$.*

Given φ, firing yields $\mathsf{Cfg}' = \langle M', S'\rangle$ *where:*

- *Consume:* $M^-(p) = M(p) \ominus \chi(p)$ *for all p.*
- *Produce: let* $\pi = \mathsf{Out}(t)(\mathsf{Cfg}, \chi, \iota)$ *with* $|\pi(p)| = W(t, p)$, *then* $M' = M^- \oplus \pi$.
- *Stack:* S' *is obtained by applying* $\mathsf{Act}(t)$ *to* S.

For a firing rule $\varphi = \langle t, \chi, \iota\rangle$, write $\mathsf{tr}(\varphi) = t$. A firing sequence is a finite sequence $\sigma = \varphi_1 \cdots \varphi_n$ such that $\mathsf{Cfg}_0[\sigma\rangle\mathsf{Cfg}_n$ where $\mathsf{Cfg}_0 = \langle M_0, \epsilon\rangle$ and the step relation follows Definition 2.

4.2 Emission and snippet well-formedness

Each transition is equipped with an emission rule that maps a firing rule to a Rust statement. API transitions emit crate calls. Structural transitions emit borrows, drops, projections, and scope delimiters. Write $\mathsf{Emit}(\sigma)$ for the snippet obtained by concatenating the emitted statements of σ.

Define closure of a configuration by stack emptiness:

$$\mathsf{Closed}(\langle M, S\rangle) \triangleq S = \epsilon.$$

A firing sequence σ is snippet-well-formed if it can be extended to reach a closed configuration:

$$\exists \sigma' : \mathsf{Cfg}_0[\sigma\cdot\sigma'\rangle\mathsf{Cfg} \ \wedge\ \mathsf{Closed}(\mathsf{Cfg}).$$

Section 5.6 defines a deterministic construction that appends a closing suffix to empty the stack.

Assumption 1 (Compiler soundness of emission). *For every enabled firing rule φ at configuration* Cfg, *the emitted statement* $\mathsf{Emit}(\varphi)$ *is accepted by the Rust type checker and borrow checker under the instantiation carried by ι. Moreover, firing φ updates marking and stack consistently with the ownership and borrowing effect represented by the emitted statement.*

Algorithm 1 SolveGuard(t, Cfg, χ)

Require: transition t, configuration $\mathsf{Cfg} = \langle M, S\rangle$, choice χ
Ensure: finite set of admissible instantiations $\mathsf{Inst}(t, \mathsf{Cfg}, \chi)$

```
Θ ← UnifyTypes(t, χ)
Λ ← UnifyLifetimes(t, χ, S)
Inst ← ∅
for all (θ, λ) ∈ Θ × Λ do
    if CheckTraitAssoc(t, θ) and CheckCap(t, Cfg, χ, θ, λ) then
        Inst ← Inst ∪ {(θ, λ, ...)}
    end if
end for
return Inst =0
```

Theorem 1 (Compiler acceptance of closed sequences). *Assume Assumption 3. Let σ be snippet-well-formed, and let $\sigma^{\downarrow}$ be obtained by appending a closing suffix that yields a closed configuration. Then* $\mathrm{Emit}(\sigma^{\downarrow})$ *is a self-contained snippet accepted by the Rust compiler under the crate context used to build $\mathcal{N}_{\mathcal{C}}$.*

Proof sketch. Each emitted statement is accepted and preserves the resource discipline by Assumption 3. Closing eliminates open scopes so the snippet does not depend on an external stack context. Thus the emitted snippet is accepted. □

4.3 Guard solving and enumeration of enabled firings

For each transition $t \in T$, the guard is written as a conjunction:

$$\begin{aligned}\mathsf{Guard}(t)(\mathsf{Cfg}, \chi, \iota) \;\equiv\; & G_t^{\mathsf{uni}}(\chi, \iota) \;\wedge\; G_t^{\mathsf{life}}(\chi, \mathsf{Cfg}.S, \iota) \\ & \wedge\; G_t^{\mathsf{trait}}(\chi, \iota) \;\wedge\; G_t^{\mathsf{cap}}(\chi, \mathsf{Cfg}.S, \iota).\end{aligned}$$

Type unification constraint G_t^{uni}. It enforces a consistent substitution for generic type variables across all consumed token colors and all type equalities induced by signatures and where-clauses.

Lifetime and stack constraint G_t^{life}. It enforces consistency of lifetime substitution and compatibility with the stack discipline. If a pop action is performed, the required pattern matches the stack top and respects last-in first-out order.

Trait and associated-type constraint G_t^{trait}. It checks all trait bounds and associated-type equalities under the chosen type substitution, requiring consistent resolution.

Capability constraint G_t^{cap}. It enforces Rust ownership and borrowing discipline at the level of places and stack frames, including freezing and blocking effects and admissibility of consuming resources under mutable operations.

4.4 Finiteness assumptions and canonicalization

The net combines colored markings, fresh identifiers, and a pushdown stack, so the state space may be infinite. The analysis domain is made finite by budgets, finite instantiation domains, and quotienting by canonicalization.

Let $B : P \to \mathbb{N}$ be a place budget. Any transition that increases the token count in p is required to satisfy $|M(p)| < B(p)$ in its guard. Let $D \in \mathbb{N}$ be a stack budget. Any push action is required to satisfy $|S| < D$ before firing.

For each generic type variable α, let $\mathsf{Cand}(\alpha) \subseteq \mathsf{GTy}$ be a finite candidate set computed from the crate signatures. Then UnifyTypes returns a finite set of substitutions.

Token colors contain value identifiers and region labels. Let $\mathsf{Canon}(\langle M, S\rangle)$ rename all value identifiers and region labels by a canonical order induced by first occurrence and canonically order multisets inside each place. Canonicalization yields representatives of α-equivalence classes induced by renaming fresh identifiers.

Assumption 2 (Finiteness of the analysis domain). *The following conditions hold.*

- **B1** *Token counts are bounded by the place budget B.*
- **B2** *Stack depth is bounded by D.*
- **B3** *Each generic variable ranges over a finite candidate set* $\mathsf{Cand}(\alpha)$*, so* SolveGuard *returns a finite set.*
- **B4** *Configurations are quotiented by canonicalization* Canon.

Theorem 2 (Finite canonical reachable set). *Under Assumption 4, the set of canonical configurations* $\widehat{\mathsf{Cfg}} = \mathsf{Canon}(\mathsf{Cfg})$ *reachable from* $\mathsf{Canon}(\langle M_0, \epsilon\rangle)$ *is finite.*

Proof sketch. Condition **B1** bounds token multiplicities per place and the place set is finite for a fixed crate. Condition **B2** bounds stack words over a finite alphabet. Condition **B3** bounds instantiation choices for each enabledness check. Condition **B4** quotients away unboundedly many fresh identifiers. Thus only finitely many canonical configurations exist. □

4.5 Canonical reachability graph and witness extraction

Definition 3 (Canonical reachability graph). *Let* $\widehat{\mathsf{Cfg}}_0 = \mathsf{Canon}(\langle M_0, \epsilon\rangle)$*. The canonical reachability graph is* $\widehat{G} = (\widehat{V}, \widehat{E})$ *where $\widehat{V}$ is the set of canonical configurations reachable from* $\widehat{\mathsf{Cfg}}_0$*, and* $(\widehat{\mathsf{Cfg}}, \varphi, \widehat{\mathsf{Cfg}}') \in \widehat{E}$ *holds iff there exist configurations* Cfg *and* Cfg' *such that* $\mathsf{Canon}(\mathsf{Cfg}) = \widehat{\mathsf{Cfg}}$, $\mathsf{Canon}(\mathsf{Cfg}') = \widehat{\mathsf{Cfg}}'$*, and* $\mathsf{Cfg}[\varphi\rangle\mathsf{Cfg}'$.

Definition 4 (Witness tree). *A witness tree is a rooted spanning tree over $\widehat{V}$ that stores, for each discovered node* $\widehat{\mathsf{Cfg}} \neq \widehat{\mathsf{Cfg}}_0$*, one parent edge* $(\widehat{\mathsf{Par}}, \varphi, \widehat{\mathsf{Cfg}}) \in \widehat{E}$*. Each node therefore admits a unique root-to-node witness firing sequence.*

Theorem 3 (Termination and completeness within bounds). *Under Assumption 4, Algorithm 9 terminates and returns the*

Algorithm 2 SaturateWithWitness($\mathcal{N}_{\mathcal{C}}$)

```
V̂ ← ∅, Ê ← ∅
parent(·) ← ⊥
Cfĝ_0 ← Canon(⟨M_0, ε⟩)
W ← {Cfĝ_0}
while W ≠ ∅ do
    remove some Cfĝ = ⟨M̂, Ŝ⟩ from W
    if Cfĝ ∈ V̂ then
        continue
    end if
    V̂ ← V̂ ∪ {Cfĝ}
    for all t ∈ T do
        for all choices χ for •t at M̂ do
            for all ι ∈ SolveGuard(t, Cfĝ, χ) do
                φ ← ⟨t, χ, ι⟩
                Cfĝ′ ← Canon(Fire(Cfĝ, φ))
                Ê ← Ê ∪ {(Cfĝ, φ, Cfĝ′)}
                if Cfĝ′ ∉ V̂ and parent(Cfĝ′) = ⊥ then
                    parent(Cfĝ′) ← (Cfĝ, φ)
                end if
                W ← W ∪ {Cfĝ′}
            end for
        end for
    end for
end while
return (V̂, Ê, parent) =0
```

complete canonical reachability graph $\widehat{G}$ together with a witness tree spanning $\widehat{V}$.

Proof sketch. The canonical reachable set is finite by Theorem 6. For each canonical configuration, the number of choices χ is finite due to bounded multiplicities. The number of instantiations is finite by Condition **B3**. Thus saturation terminates. Completeness follows from enumerating all enabled firing rules at each discovered configuration. □

4.6 Graph analyses on the bounded reachability graph

For a transition t, define the set of nodes where t is enabled by some firing rule:

$$\mathsf{En}(t) = \{\, \widehat{\mathsf{Cfg}} \in \widehat{V} \mid \exists \varphi, \widehat{\mathsf{Cfg}}' \;:\; (\widehat{\mathsf{Cfg}}, \varphi, \widehat{\mathsf{Cfg}}') \in \widehat{E} \wedge \mathsf{tr}(\varphi) = t \,\}.$$

Definition 5 (Dead, quasi-live, live). *Let $t \in T$.*

- *t is* dead *iff* $\mathsf{En}(t) = \emptyset$.
- *t is* quasi-live *iff* $\mathsf{En}(t) \neq \emptyset$.
- *t is* live *iff for every $\widehat{\mathsf{Cfg}} \in \widehat{V}$ there exists a path in $\widehat{G}$ from $\widehat{\mathsf{Cfg}}$ to some node in* $\mathsf{En}(t)$.

A canonical configuration $\widehat{\mathsf{Cfg}}$ is a deadlock iff it has no outgoing edge in $\widehat{E}$. Deadlocks are detected by scanning out-degrees in $\widehat{G}$.

Algorithm 3 AnalyzeDeadQuasiLive($\widehat{G}$)

```
Require: Ĝ = (V̂, Ê)
Ensure: Dead, QuasiLive ⊆ T
Seen ← ∅
for all (Cfĝ, φ, Cfĝ′) ∈ Ê do
    Seen ← Seen ∪ {tr(φ)}
end for
QuasiLive ← Seen
Dead ← T \ Seen
return (Dead, QuasiLive) =0
```

Algorithm 4 AnalyzeLiveness($\widehat{G}, t$)

```
Require: Ĝ = (V̂, Ê), transition t
Ensure: boolean IsLive(t)
Targets ← En(t)
if Targets = ∅ then
    return false
end if
build reverse adjacency Ê^{-1}
R ← Targets
Q ← Targets
while Q ≠ ∅ do
    pop Cfĝ from Q
    for all (Cfĝ_pre, _, Cfĝ) ∈ Ê^{-1} do
        if Cfĝ_pre ∉ R then
            R ← R ∪ {Cfĝ_pre}
            Q ← Q ∪ {Cfĝ_pre}
        end if
    end for
end while
return (R = V̂) =0
```

Theorem 4 (Decidability within the bounded domain). *Under Assumption 4, reachability of a canonical configuration, deadness, quasi-liveness, liveness, and deadlock detection are decidable by finite graph algorithms on $\widehat{G}$.*

Proof sketch. The graph $\widehat{G}$ is finite by Theorem 6. Each property is reducible to reachability or reverse reachability on a finite directed graph. Thus the analyses terminate and are decidable. □

4.7 Closing the stack and obtaining self-contained snippets

Lemma 1 (Closing terminates). *If* CloseStack *does not get stuck, it terminates in at most $|S|$ iterations and returns a sequence that empties the stack.*

Proof sketch. Each iteration strictly decreases the stack length by at least one pop action. □

Algorithm 5 CloseStack(Cfg)

Require: reachable configuration $\mathrm{Cfg} = \langle M, S\rangle$
Ensure: a closing sequence σ_{close} such that $\mathrm{Cfg}[\sigma_{\mathrm{close}}\rangle\langle M', \epsilon\rangle$

$\sigma_{\mathrm{close}} \leftarrow \epsilon$
while $S \neq \epsilon$ **do**
  let $\gamma = \mathrm{top}(S)$
  choose the unique applicable ending transition t_{end} for γ by its stack pattern
  choose χ matching the required input tokens for t_{end}
  pick some $\iota \in \mathrm{SolveGuard}(t_{\mathrm{end}}, \langle M, S\rangle, \chi)$
  fire $\varphi = \langle t_{\mathrm{end}}, \chi, \iota\rangle$ to update (M, S)
  $\sigma_{\mathrm{close}} \leftarrow \sigma_{\mathrm{close}} \cdot \varphi$
end while
return σ_{close} =0

Algorithm 6 FindWitness($\widehat{G}, \Phi$)

Require: canonical reachability graph $\widehat{G} = (\widehat{V}, \widehat{E})$ with parent map parent, goal predicate Φ
Ensure: a witness firing sequence or report unreachable

Targets $\leftarrow \{\, \widehat{\mathrm{Cfg}} \in \widehat{V} \mid \Phi(\widehat{\mathrm{Cfg}}) \,\}$
if Targets $= \emptyset$ **then**
  return unreachable
end if
choose some $\widehat{\mathrm{Cfg}}^{\star} \in$ Targets
extract σ by backtracking parent pointers from $\widehat{\mathrm{Cfg}}^{\star}$ to $\widehat{\mathrm{Cfg}}_0$
return σ =0

Given a firing sequence σ, let Cfg_σ denote the reached configuration. Define the closed sequence $\sigma^{\downarrow} = \sigma{\cdot}\mathrm{CloseStack}(\mathrm{Cfg}_\sigma)$. The snippet $\mathrm{Emit}(\sigma^{\downarrow})$ is self-contained by definition of Closed.

4.8 Goal reachability and witness-based synthesis

Many synthesis tasks reduce to reachability of a goal predicate over canonical configurations. Let Φ be a predicate over canonical configurations. A witness sequence for Φ is a firing sequence σ such that $\widehat{\mathrm{Cfg}}_0[\sigma\rangle\widehat{\mathrm{Cfg}}$ and $\Phi(\widehat{\mathrm{Cfg}})$ holds.

If the goal additionally requires self-containment, the predicate Φ can include Closed. The emitted snippet is then obtained from the closed sequence $\sigma^{\downarrow}$.

4.9 Large language model assistance for identifier synthesis

The net semantics and the firing sequence determine the type and borrow structure of the synthesized program. However, the surface form of Rust code still requires concrete identifiers and stylistic choices. To avoid encoding naming heuristics into places and token colors, the emission step is decomposed into a typed skeleton and a semantics-preserving renaming pass.

Algorithm 7 LLMNameFill(σ)

Require: closed firing sequence $\sigma^{\downarrow}$
Ensure: a readable snippet with validated naming

$c \leftarrow \mathrm{EmitCore}(\sigma^{\downarrow})$
extract a typing summary Γ from the firing sequence, including each placeholder and its type form
query an LLM with c and Γ to obtain a candidate renaming map ρ
if ρ violates scope, injectivity, or reserved keyword constraints **then**
  $\rho \leftarrow$ the identity map
end if
$c' \leftarrow \mathrm{Rename}(c, \rho)$
return c' =0

Let $\mathrm{EmitCore}(\sigma)$ be the core emitter that produces a compiler-accepted skeleton using a fixed placeholder naming scheme. A typical scheme assigns fresh identifiers $v_1, v_2, \ldots$ to produced values in first-occurrence order and reuses them according to the dataflow induced by σ. Let ρ be a renaming map from placeholders to user-level identifiers that respects lexical scoping and avoids reserved keywords. Write $\mathrm{Rename}(c, \rho)$ for the capture-avoiding renaming applied to code c.

Lemma 2 (Renaming preserves compiler acceptance). *Let $c = \mathrm{EmitCore}(\sigma^{\downarrow})$ be a self-contained skeleton. If ρ is capture-avoiding and scope-respecting, then* $\mathrm{Rename}(c, \rho)$ *is accepted by the Rust compiler in the same crate context.*

Proof sketch. Capture-avoiding renaming is an α-conversion that preserves binding structure. The typing and borrowing constraints are preserved because only identifiers are changed and all references remain bound to the same definitions. Thus compiler acceptance is preserved. □

The term large language model is abbreviated as LLM in the remainder. The LLM is used only to propose a renaming map ρ together with optional comments. The proposal is filtered by syntactic constraints and by a deterministic scope check. If the proposal fails the checks, the system falls back to the placeholder scheme.

This separation assigns semantic responsibility to the Petri net analysis and assigns readability responsibility to the naming pass. The soundness argument reduces to Lemma 2 because the LLM is not permitted to change the control, dataflow, or borrow structure determined by σ.

5 Formal Analysis for Snippet Synthesis from a Pushdown Colored Petri Net

This section establishes the analysis basis for generating compiler-accepted Rust snippets from the pushdown colored Petri net

$\mathcal{N}_{\mathcal{C}}$. Places encode ownership and borrow capabilities, and the pushdown stack encodes structured scope information that must be discharged to obtain self-contained snippets. The analysis proceeds by saturating a canonical reachability graph under boundedness and canonicalization assumptions, then extracting firing sequences as witnesses for coverage objectives and user-specified objectives.

5.1 Configurations, enabledness, and firing sequences

A configuration is a pair $\mathsf{Cfg} = \langle M, S\rangle$ where M is a marking over places and S is a stack word. The marking records available symbolic resources, and the stack records borrow and projection scopes that are opened by structural transitions and must be closed before snippet emission.

A firing sequence is a finite sequence of firing rules $\sigma = \varphi_1 \cdots \varphi_n$ such that $\mathsf{Cfg}_0[\sigma\rangle\mathsf{Cfg}_n$ where $\mathsf{Cfg}_0 = \langle M_0, \epsilon\rangle$ and $\mathsf{Cfg}[\varphi\rangle\mathsf{Cfg}'$ follows Definition 2. We write $\mathsf{Cfg}[\epsilon\rangle\mathsf{Cfg}$ for the empty sequence, and we extend $[\cdot\rangle$ to sequences by repeated firing.

Each transition is equipped with an emission rule that maps the firing rule to a Rust statement. We write $\mathsf{Emit}(\sigma)$ for the Rust snippet synthesized from σ by concatenating the emitted statements. API transitions emit crate calls, and structural transitions emit borrows, drops, projections, and reborrow scopes.

Define:

$$\mathsf{Closed}(\langle M, S\rangle) \triangleq S = \epsilon.$$

A firing sequence σ is *snippet-well-formed* if it can be extended to reach a closed configuration:

$$\exists \sigma' : \mathsf{Cfg}_0[\sigma \cdot \sigma'\rangle\mathsf{Cfg} \wedge \mathsf{Closed}(\mathsf{Cfg}).$$

Section 5.6 gives a deterministic closing construction that appends a suffix sequence to empty the stack.

Assumption 3. *For every enabled firing rule φ at configuration* Cfg*, the emitted statement* $\mathsf{Emit}(\varphi)$ *is accepted by the Rust type checker and borrow checker under the instantiation carried by ι. Moreover, firing φ updates marking and stack consistently with the ownership and borrowing effect represented by the emitted statement.*

Theorem 5 (Compiler acceptance of closed sequences). *Assume Assumption 3. Let σ be snippet-well-formed, and let $\sigma^{\downarrow}$ be obtained by appending a closing suffix that yields a closed configuration. Then* $\mathsf{Emit}(\sigma^{\downarrow})$ *is a self-contained snippet accepted by the Rust compiler under the crate context used to build $\mathcal{N}_{\mathcal{C}}$.*

Proof sketch. Each emitted statement is accepted, and the net effect respects the resource discipline by Assumption 3. Closing eliminates open scopes so the snippet does not rely on an external stack context. Thus the emitted snippet is accepted. □

Algorithm 8 SolveGuard(t, Cfg, χ)

Require: transition t, configuration $\mathsf{Cfg} = \langle M, S\rangle$, choice χ
Ensure: finite set of admissible instantiations $\mathsf{Inst}(t, \mathsf{Cfg}, \chi)$

```
Θ ← UnifyTypes(t, χ)
Λ ← UnifyLifetimes(t, χ, S)
Inst ← ∅
for all (θ, λ) ∈ Θ × Λ do
    if CheckTraitAssoc(t, θ) and CheckCap(t, Cfg, χ, θ, λ) then
        Inst ← Inst ∪ {(θ, λ, ...)}
    end if
end for
return Inst =0
```

5.2 Guard solving and enumeration of enabled firings

For each transition $t \in T$, the guard is written as a conjunction:

$$\mathsf{Guard}(t)(\mathsf{Cfg}, \chi, \iota) \equiv G_t^{\mathsf{uni}}(\chi, \iota) \wedge G_t^{\mathsf{life}}(\chi, \mathsf{Cfg}.S, \iota) \wedge G_t^{\mathsf{trait}}(\chi, \iota) \wedge G_t^{\mathsf{cap}}(\chi, \mathsf{Cfg}.S, \iota).$$

Type unification constraint G_t^{uni}. It enforces a consistent substitution for generic type variables across all consumed token colors and all type equalities induced by signatures and where-clauses. The instantiation record ι contains a substitution θ. The constraint requires agreement between θ and token-carried substitutions and requires all signature equalities to hold after applying θ.

Lifetime and stack constraint G_t^{life}. It enforces consistency of lifetime substitution and compatibility with the stack discipline. If the same lifetime variable occurs in multiple parameters, the chosen tokens agree on the lifetime substitution. If t performs a pop action, the required pattern matches the stack top and respects last-in first-out order.

Trait and associated-type constraint G_t^{trait}. It checks all trait bounds and associated-type equalities in the where-clause under θ, requiring consistent resolution.

Capability constraint G_t^{cap}. It enforces Rust ownership and borrowing discipline at the level of places and stack frames, including freezing and blocking effects and the admissibility of consuming resources under mutable operations.

5.3 Finiteness assumptions and canonicalization

The net combines colored markings, fresh identifiers, and a pushdown stack, so the state space may be infinite. The analysis domain is made finite by budgets, finite instantiation domains, and quotienting by canonicalization.

Let $B : P \to \mathbb{N}$ be a place budget. Any transition that increases the token count in p is required to satisfy $|M(p)| < B(p)$ in its guard. This condition applies to constructors, duplication effects, and calls that return fresh owned values. Let

$D \in \mathbb{N}$ be a stack budget. Any push action is required to satisfy $|S| < D$ before firing.

For each generic type variable α, let $\mathsf{Cand}(\alpha) \subseteq \mathsf{GTy}$ be a finite candidate set computed from the crate signatures. Then UnifyTypes returns a finite set of substitutions.

Token colors contain value identifiers and region labels. Let $\mathsf{Canon}(\langle M, S\rangle)$ rename all value identifiers and region labels by a canonical order induced by first occurrence and canonically order multisets inside each place. Canonicalization yields representatives of α-equivalence classes induced by renaming fresh identifiers.

Assumption 4 (Finiteness of the analysis domain). *We assume the following conditions.*

- **B1** *Token counts are bounded by the place budget B.*
- **B2** *Stack depth is bounded by D.*
- **B3** *Each generic variable ranges over a finite candidate set* $\mathsf{Cand}(\alpha)$*, so* SolveGuard *returns a finite set.*
- **B4** *Configurations are quotiented by canonicalization* Canon.

Theorem 6 (Finite canonical reachable set). *Under Assumption 4, the set of canonical configurations* $\widehat{\mathsf{Cfg}} = \mathsf{Canon}(\mathsf{Cfg})$ *reachable from* $\mathsf{Canon}(\langle M_0, \epsilon\rangle)$ *is finite.*

Proof sketch. Condition B1 bounds token multiplicities per place and the place set is finite for a fixed crate. Condition B2 bounds stack words over a finite alphabet. Condition B3 bounds instantiation choices for each enabledness check. Condition B4 quotients away unboundedly many fresh identifiers. Thus only finitely many canonical configurations exist. □

5.4 Canonical reachability graph and witness extraction

Definition 6 (Canonical reachability graph). *Let* $\widehat{\mathsf{Cfg}}_0 = \mathsf{Canon}(\langle M_0, \epsilon\rangle)$*. The canonical reachability graph is* $\widehat{G} = (\widehat{V}, \widehat{E})$ *where* $\widehat{V}$ *is the set of canonical configurations reachable from* $\widehat{\mathsf{Cfg}}_0$*, and an edge* $(\widehat{\mathsf{Cfg}}, \varphi, \widehat{\mathsf{Cfg}}') \in \widehat{E}$ *holds iff there exist configurations* Cfg *and* Cfg' *such that* $\mathsf{Canon}(\mathsf{Cfg}) = \widehat{\mathsf{Cfg}}, \mathsf{Canon}(\mathsf{Cfg}') = \widehat{\mathsf{Cfg}}'$*, and* $\mathsf{Cfg}[\varphi\rangle\mathsf{Cfg}'$.

Definition 7 (Witness tree). *A witness tree is a rooted spanning tree over* $\widehat{V}$ *that stores, for each discovered node* $\widehat{\mathsf{Cfg}} \neq \widehat{\mathsf{Cfg}}_0$*, one parent edge* $(\widehat{\mathsf{Par}}, \varphi, \widehat{\mathsf{Cfg}}) \in \widehat{E}$*. Each node therefore admits a unique root-to-node witness firing sequence.*

Theorem 7 (Termination and completeness). *Under Assumption 4, Algorithm 9 terminates and returns the complete canonical reachability graph* $\widehat{G}$ *together with a witness tree spanning* $\widehat{V}$*.*

Proof sketch. The canonical reachable set is finite by Theorem 6. For each canonical configuration, the number of choices χ is finite due to bounded multiplicities. The number of instantiations is finite by Condition B3. Thus saturation terminates. Completeness follows from enumerating all enabled firing rules at each discovered configuration. □

Algorithm 9 SaturateWithWitness($\mathcal{N}_{\mathcal{C}}$)

```
1:  V̂ ← ∅, Ê ← ∅
2:  parent(·) ← ⊥
3:  Cfĝ_0 ← Canon(⟨M_0, ε⟩)
4:  W ← {Cfĝ_0}
5:  while W ≠ ∅ do
6:      remove some Cfĝ = ⟨M̂, Ŝ⟩ from W
7:      if Cfĝ ∈ V̂ then
            continue
8:      end if
9:      V̂ ← V̂ ∪ {Cfĝ}
10:     for all t ∈ T do
11:         for all choices χ for •t at M̂ do
12:             for all ι ∈ SolveGuard(t, Cfĝ, χ) do
13:                 φ ← ⟨t, χ, ι⟩
14:                 Cfĝ' ← Canon(Fire(Cfĝ, φ))
15:                 Ê ← Ê ∪ {(Cfĝ, φ, Cfĝ')}
16:                 if Cfĝ' ∉ V̂ and parent(Cfĝ') = ⊥ then
17:                     parent(Cfĝ') ← (Cfĝ, φ)
18:                 end if
19:                 W ← W ∪ {Cfĝ'}
20:             end for
21:         end for
22:     end for
23: end while
24: return (V̂, Ê, parent) =0
```

Let $\mathsf{Ch}(\widehat{M}, t)$ be the number of valid choices for t at marking $\widehat{M}$, and let $\mathsf{Inst}(\widehat{\mathsf{Cfg}}, t, \chi)$ be the number of admissible instantiations. The worst-case saturation cost is

$$O\left(\sum_{\widehat{\mathsf{Cfg}} \in \widehat{V}} \sum_{t \in T} \sum_{\chi} |\mathsf{Inst}(\widehat{\mathsf{Cfg}}, t, \chi)|\right).$$

5.5 Deadness, liveness, and deadlocks on the reachability graph

For a transition t, define the set of nodes where t is enabled by some firing rule:

$$\mathsf{En}(t) = \{\, \widehat{\mathsf{Cfg}} \in \widehat{V} \mid \exists \varphi, \widehat{\mathsf{Cfg}}' \;:\; (\widehat{\mathsf{Cfg}}, \varphi, \widehat{\mathsf{Cfg}}') \in \widehat{E} \wedge \varphi.t = t \,\}.$$

Definition 8 (Dead, quasi-live, live). *Let* $t \in T$*.*

- t *is* dead *iff* $\mathsf{En}(t) = \emptyset$*.*
- t *is* quasi-live *iff* $\mathsf{En}(t) \neq \emptyset$*.*
- t *is* live *iff for every* $\widehat{\mathsf{Cfg}} \in \widehat{V}$ *there exists a path in* $\widehat{G}$ *from* $\widehat{\mathsf{Cfg}}$ *to some node in* $\mathsf{En}(t)$*.*

A canonical configuration $\widehat{\mathsf{Cfg}}$ is a *deadlock* iff it has no outgoing edge in $\widehat{E}$. Deadlocks are detected by scanning the out-degree of each node in $\widehat{G}$.

Algorithm 10 AnalyzeDeadQuasiLive($\widehat{G}$)

Require: $\widehat{G} = (\widehat{V}, \widehat{E})$
Ensure: Dead, QuasiLive $\subseteq T$

```
Seen ← ∅
for all (Cfĝ, φ, Cfĝ′) ∈ Ê do
    Seen ← Seen ∪ {φ.t}
end for
QuasiLive ← Seen
Dead ← T \ Seen
return (Dead, QuasiLive) =0
```

Algorithm 11 AnalyzeLiveness($\widehat{G}, t$)

Require: $\widehat{G} = (\widehat{V}, \widehat{E})$, transition t
Ensure: boolean IsLive(t)

```
Targets ← En(t)
if Targets = ∅ then
    return false
end if
build reverse adjacency Ê^{-1}
R ← Targets
Q ← Targets
while Q ≠ ∅ do
    pop Cfĝ from Q
    for all (Cfĝ_pre, _, Cfĝ) ∈ Ê^{-1} do
        if Cfĝ_pre ∉ R then
            R ← R ∪ {Cfĝ_pre}
            Q ← Q ∪ {Cfĝ_pre}
        end if
    end for
end while
return (R = V̂) =0
```

Algorithm 12 CloseStack(Cfg)

Require: reachable configuration Cfg $= \langle M, S\rangle$
Ensure: a closing sequence σ_{close} such that Cfg$[\sigma_{\text{close}}\rangle\langle M', \epsilon\rangle$

```
σ_close ← ε
while S ≠ ε do
    let γ = top(S)
    choose the unique applicable ending transition t_end for γ by its stack pattern
    choose χ matching the required input tokens for t_end
    pick some ι ∈ SolveGuard(t_end, ⟨M, S⟩, χ)
    fire φ = ⟨t_end, χ, ι⟩ to update (M, S)
    σ_close ← σ_close · φ
end while
return σ_close =0
```

5.6 Closing the stack and obtaining self-contained snippets

Lemma 3 (Closing terminates). *If* CloseStack *does not get stuck, it terminates in at most* $|S|$ *iterations and returns a sequence that empties the stack.*

Proof sketch. Each iteration strictly decreases the stack length by at least one pop action. □

Given a firing sequence σ, let Cfg_σ denote the reached configuration. The closed sequence is defined by $\sigma^{\downarrow} = \sigma{\cdot}\text{CloseStack}(\text{Cfg}_\sigma)$. The snippet $\text{Emit}(\sigma^{\downarrow})$ is self-contained by construction of Closed.

5.7 Coverage metrics over sets of snippets

Let $T_{\text{API}} \subseteq T$ be the subset of transitions corresponding to crate functions and methods. Define the reachable API transition set:

$$T_{\text{API}}^{\text{reach}} = \{\, t \in T_{\text{API}} \mid \mathsf{En}(t) \neq \emptyset \,\}.$$

Let $P_{\text{form}} \subseteq P$ be places representing type forms of interest. Define:

$$P_{\text{form}}^{\text{reach}} = \{\, p \in P_{\text{form}} \mid \exists \widehat{\text{Cfg}} = \langle \widehat{M}, \hat{S}\rangle \in \widehat{V} \;:\; \widehat{M}(p) \neq \emptyset \,\}.$$

For borrow shapes, define:

$$\mathcal{B}^{\text{reach}} = \{\, \mathsf{Shape}(\hat{S}) \mid \langle \widehat{M}, \hat{S}\rangle \in \widehat{V} \,\}.$$

Let Σ be a finite set of firing sequences. Define:

$$\mathsf{API}(\Sigma) = \bigcup_{\sigma\in\Sigma} \{\, \varphi.t \mid \varphi \in \sigma,\ \varphi.t \in T_{\text{API}} \,\}.$$

Let σ induce intermediate configurations $\text{Cfg}_i = \langle M_i, S_i\rangle$. Define:

$$\mathsf{Vis}_P(\Sigma) = \bigcup_{\sigma\in\Sigma}\bigcup_i \{\, p \in P_{\text{form}} \mid M_i(p) \neq \emptyset \,\}, \qquad \mathsf{Vis}_B(\Sigma) = \bigcup_{\sigma\in\Sigma}\bigcup_i \{\mathsf{Shape}(S_i)\}.$$

Then:

$$\text{Cov}_{\text{API}}(\Sigma) = \frac{|\mathsf{API}(\Sigma)|}{|T_{\text{API}}^{\text{reach}}|},$$

$$\text{Cov}_{\text{Type}}(\Sigma) = \frac{|\mathsf{Vis}_P(\Sigma) \cap P_{\text{form}}^{\text{reach}}|}{|P_{\text{form}}^{\text{reach}}|},$$

$$\text{Cov}_{\text{Borrow}}(\Sigma) = \frac{|\mathsf{Vis}_B(\Sigma)|}{|\mathcal{B}^{\text{reach}}|}.$$

We combine them as:

$$\text{GCov}(\Sigma) = w_1{\cdot}\text{Cov}_{\text{API}}(\Sigma) + w_2{\cdot}\text{Cov}_{\text{Type}}(\Sigma) + w_3{\cdot}\text{Cov}_{\text{Borrow}}(\Sigma), \qquad w_i \geq 0.$$

5.8 Witness-based synthesis for coverage objectives

Using the parent map of the witness tree, each reachable node $\widehat{\text{Cfg}}$ admits a unique witness firing sequence $\text{WitNode}(\widehat{\text{Cfg}})$ obtained by backtracking parent pointers to the root. For each edge $(\widehat{\text{Cfg}}, \varphi, \widehat{\text{Cfg}}') \in \widehat{E}$, define:

$$\text{WitEdge}(\widehat{\text{Cfg}}, \varphi, \widehat{\text{Cfg}}') = \text{WitNode}(\widehat{\text{Cfg}}) \cdot \varphi.$$

Theorem 8 (Full model-level coverage within bounds). *Let* $(\widehat{V}, \widehat{E}, \text{parent})$ *be the output of Algorithm 9. Define:*

$$\Sigma_{\text{API}} = \{\, \text{WitEdge}(e) \mid e \in \widehat{E},\ e.\varphi.t \in T_{\text{API}} \,\}.$$

Algorithm 13 GreedyCover($\mathcal{S}, m$)

```
Σ ← ∅, U ← all objectives initially uncovered
for i = 1 to m do
    choose σ* ∈ S \ Σ maximizing |Obj(σ*) ∩ U|
    Σ ← Σ ∪ {σ*}
    U ← U \ Obj(σ*)
end for
return Σ =0
```

Then $\mathrm{Cov}_{\mathrm{API}}(\Sigma_{\mathrm{API}}) = 1$. *Moreover, by adding witnesses for first occurrences of each* $p \in P^{\mathrm{reach}}_{\mathrm{form}}$ *and each* $b \in \mathcal{B}^{\mathrm{reach}}$, *we obtain a finite set* $\Sigma^\star$ *such that*

$$\mathrm{Cov}_{\mathrm{API}}(\Sigma^\star) = \mathrm{Cov}_{\mathrm{Type}}(\Sigma^\star) = \mathrm{Cov}_{\mathrm{Borrow}}(\Sigma^\star) = 1.$$

Proof sketch. For any $t \in T^{\mathrm{reach}}_{\mathrm{API}}$, there exists an edge in $\widehat{E}$ labeled by a firing rule whose transition is t. The corresponding witness edge sequence contains t. The place and shape statements follow by selecting a node or edge where the objective first appears. □

For each witness firing sequence σ, we emit the closed sequence $\sigma^{\downarrow}$ and obtain the corresponding snippet $\mathrm{Emit}(\sigma^{\downarrow})$.

6 Evaluation

6.1 Research Questions

We structure the evaluation around the following research questions:

RQ1 (Compilability & Executability). Does our PCPN-guided generator produce compilable and executable harnesses at a higher rate than baselines that do not model ownership/borrowing/lifetimes/generics precisely?

RQ4 (Feature Support and Stress Cases). How robust is the approach on crates that heavily use (i) borrow-carrying types and explicit lifetimes in signatures, (ii) generic APIs with trait bounds and associated types, and (iii) composite types requiring field projection/reborrowing?

RQ5 (Ablation: Which Components Matter?). What is the contribution of each major component (pushdown lifetime stack, generic partial-order instantiation, field (re)borrowing rules, POR/heuristics if any) to compilability and coverage?

RQ6 (Model Coverage → Code Coverage). To what extent does increasing *net-level coverage* (e.g., transition coverage over API transitions) correlate with increased *target-code* coverage, and can it serve as a reliable surrogate objective?

6.2 Benchmarks and Subjects

Real-world crates. We evaluate on a curated set of Rust crates drawn from crates.io. We select crates that (i) expose non-trivial public APIs (multiple modules/types), (ii) include stateful protocols (resources/handles/transactions/builders), and (iii) collectively cover diverse domains (parsers, codecs, data structures, storage, networking, etc.). For each crate, we extract callable items $\mathcal{F}$ (public functions/methods) and their signatures via the compiler front-end.

Feature-stratified subsets. We partition crates into groups based on signature-level features:

- **Lifetime-heavy**: explicit lifetimes and borrow-carrying return types in signatures;
- **Generic-heavy**: frequent type parameters with trait bounds / where-clauses / associated types;
- **Composite-heavy**: deep nested ADTs and frequent field projections (struct/tuple) in APIs.

6.3 Baselines

We compare against the following baselines:

B0: Byte-oriented fuzzing harness. A conventional setup where the fuzzer mutates a byte array and a harness decodes it into inputs for a fixed entry point.

B1: Signature-driven random sequences (no borrow/lifetime model). A type-directed generator that samples call sequences using signatures, but treats values as freely duplicable or ends borrows heuristically without a formal stack discipline.

B2: Petri net without pushdown (finite-state approximation). A Petri-net-based generator that uses places/transitions derived from signatures but replaces the pushdown lifetime mechanism with a finite-state approximation (e.g., lexical-only or bounded non-nested borrow tracking).

B3 (optional): Property-based testing. A property-based generator (e.g., arbitrary structured values) for selected APIs, without stateful call-sequence generation.

6.4 Metrics

Compilability and executability. Let $\mathcal{H}$ be the set of synthesized harnesses. We define:

$$\mathbf{CHR} = \frac{|\{h \in \mathcal{H} \mid \mathrm{rustc}(h) \text{ accepts}\}|}{|\mathcal{H}|}, \tag{1}$$

$$\mathbf{EHR} = \frac{|\{h \in \mathcal{H} \mid h \text{ executes and completes initialization}\}|}{|\mathcal{H}|}. \tag{2}$$

Target-library coverage. For each crate-under-test, we measure standard code coverage (e.g., line and branch coverage):

$$\mathbf{TLC} = \frac{\text{covered lines in target crate}}{\text{total instrumented lines in target crate}}, \tag{3}$$

$$\mathbf{TBC} = \frac{\text{covered branches in target crate}}{\text{total instrumented branches in target crate}}. \tag{4}$$

Model (net-level) coverage. Let $T_{\mathsf{api}} \subseteq T$ be transitions corresponding to extracted API calls. For a trace multiset Σ (generated during exploration/fuzzing), define:

$$\mathbf{NTC} = \frac{|\{t \in T_{\mathsf{api}} \mid \exists \sigma \in \Sigma : t \in \sigma\}|}{|T_{\mathsf{api}}|}. \tag{5}$$

Table 1: Benchmark summary (to be filled).

Crate	LOC	#API items $\lvert\mathcal{F}\rvert$	Lifetime-heavy?	Generic-heavy?

We record: (i) the number of unique failures (deduplicated by stack trace hash or sanitizer report signature), and (ii) time-to-first-crash (TTFC).

Cost and scalability. We measure: net construction time, exploration time per trace, harness compilation time, and fuzzing throughput (exec/s).

6.5 Experimental Protocol

For each crate and each approach (ours and baselines), we perform the following steps:

(1) **Model extraction (ours / B2).** Extract signatures $\mathcal{F}$ and build the Petri-net instance.
(2) **Harness generation.** Generate K harnesses (or K traces), each bounded by max trace length L and stack depth D.
(3) **Compilation filter.** Compile each harness; record CHR and compile-time.
(4) **Fuzzing run.** Run each compiled harness for a fixed time budget B_{time} (or fixed total executions), using the same fuzzer backend and identical resource limits.
(5) **Measurement.** Collect target coverage (TLC/TBC), net-level coverage (NTC), and failures.

Repetitions and statistics. Each configuration is repeated R times with independent random seeds. We report mean and standard deviation (or median and IQR) and use non-parametric tests when appropriate.

6.6 Ablation and Sensitivity Studies

Ablations. We evaluate the following variants:

- **NoStack**: remove the pushdown lifetime stack and replace with a finite approximation;
- **NoPO**: replace the partial-order-based generic instantiation with random instantiation from the same candidate pool;
- **NoFieldReborrow**: disable field projection/reborrow transitions;
- **NoAssoc**: disable associated-type rewriting transitions.

We compare each variant against the full system in CHR/EHR, TLC/TBC, and TTFC.

Sensitivity. We vary L (trace length bound), D (stack depth bound), and $\lvert\mathsf{Cand}(\alpha)\rvert$ (generic candidate pool size) to quantify trade-offs between exploration power, compilation success, and runtime coverage.

6.7 Reporting Format

We summarize results using: (i) per-crate coverage plots (TLC/TBC over time), (ii) bar charts for CHR/EHR across approaches, (iii) scatter plots for NTC vs. TLC (RQ6), and (iv) tables listing unique crashes and TTFC.

6.8 Threats to Validity

We discuss common threats: (i) benchmark representativeness (external validity), (ii) fairness of baseline implementations and harness effort (internal validity), and (iii) measurement bias in coverage instrumentation and crash deduplication (construct validity).

References